VERIQTA

PRODUCTION READY PRACTITIONER GUIDE

Amazon EKS HANDBOOK

BUILD • DEPLOY • SECURE • OPERATE

- PRODUCTION ARCHITECTURE
- SECURITY & IAM
- SCALING & PERFORMANCE
- AUTOMATION & GITOPS
- MONITORING & OBSERVABILITY
- STORAGE & NETWORKING
- RESILIENCE & DISASTER RECOVERY

20 PAGES
COMPLETE CURRICULUM FOR DEVOPS & PLATFORM ENGINEERS

BEST PRACTICES | SECURE BY DESIGN | HIGH AVAILABILITY | COST OPTIMIZED

/VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA

VERIQTA

AMAZON EKS HANDBOOK

EKS PRODUCTION ARCHITECTURE

EKS CONTROL PLANE

- Fully managed by AWS
- Runs the Kubernetes API Server, etcd, Scheduler, Controller Manager
- Highly available across multiple AZs
- You do not manage or access control plane nodes

WORKER NODES

- Run your containerized applications
- Join the cluster and register with the control plane
- Can be managed node groups, self-managed nodes, or Fargate

MANAGED NODE GROUPS

- AWS manages EC2 instances for you
- Automatic scaling, patching, and upgrades
- Support for Spot and On-Demand instances

FARGATE

- Serverless compute for containers
- No servers to manage
- Pay for vCPU and memory per workload
- Ideal for batch, microservices, and spiky workloads

VPC ARCHITECTURE

- EKS runs inside your Amazon VPC
- Use private subnets for worker nodes and Fargate
- Use public subnets for load balancers and NAT Gateways
- Route tables, Security Groups, and NACLs control network traffic

PUBLIC VS PRIVATE API ENDPOINTS

PUBLIC ENDPOINT

- Accessible over the internet
- Use with CIDR allow-list
- Easier access for dev/ops

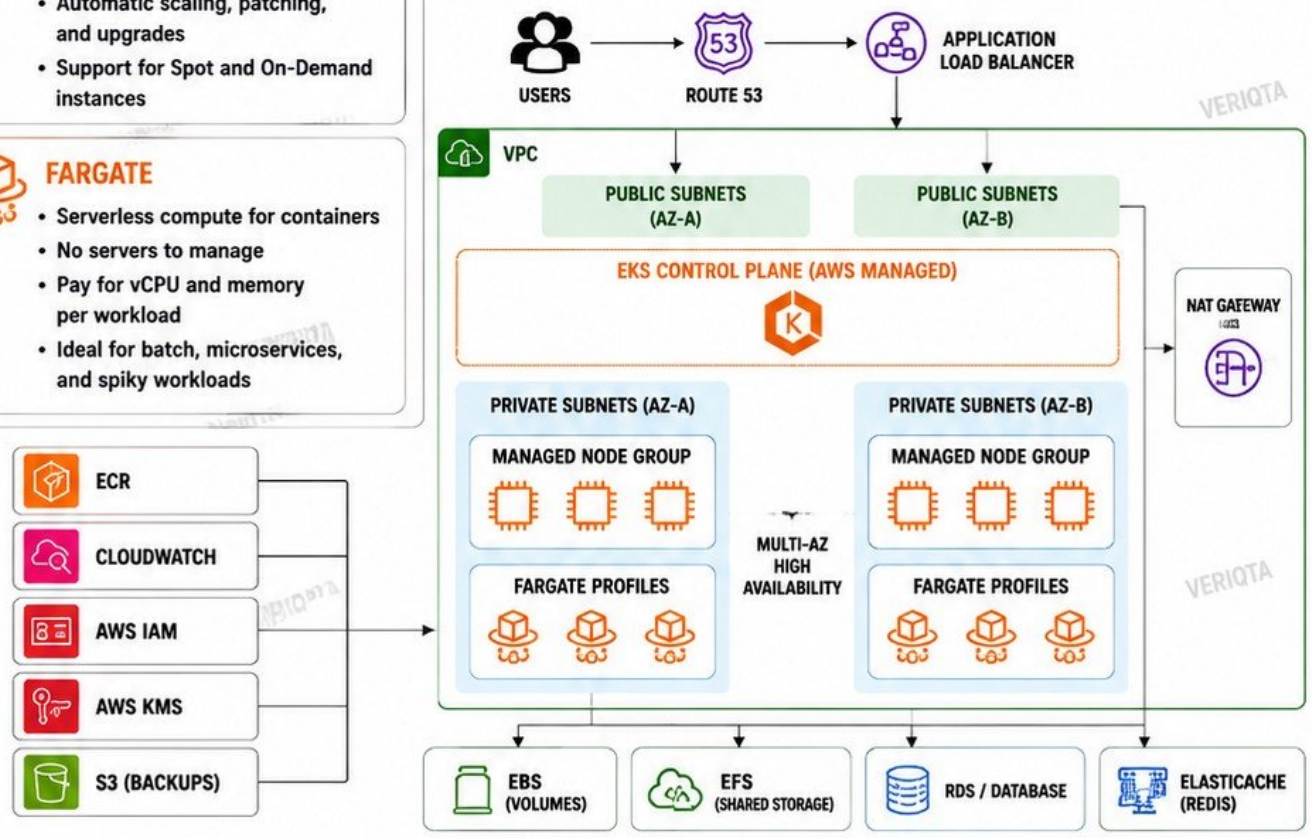
PRIVATE ENDPOINT

- Accessible only inside the VPC or via Direct Connect / VPN
- More secure for production
- Recommended for private clusters

MULTI-AZ DESIGN

- Spread worker nodes across multiple Availability Zones
- Ensures high availability and fault tolerance
- Survives AZ failures without impacting workloads
- Use Multi-AZ load balancers and zonal awareness

PRODUCTION ARCHITECTURE DIAGRAM



HIGHLY AVAILABLE • SECURE • SCALABLE • MANAGED • PRODUCTION READY

VERIQTA

BUILDING A PRODUCTION EKS CLUSTER

1 CLUSTER PLANNING

- Define use case and workloads
- Identify requirements and constraints
- Plan for high availability and scalability
- Choose networking, security, and access model
- Estimate capacity and cost

2 KUBERNETES VERSIONS

- EKS supports multiple Kubernetes versions
- Use a supported version for production
- Enable automatic minor version upgrades
- Check add-on compatibility
- Plan upgrade and rollback strategy

3 VPC AND SUBNET REQUIREMENTS

- VPC with at least 2 Availability Zones
- Private subnets for worker nodes and Fargate
- Public subnets for load balancers (optional)
- NAT Gateway for outbound internet access
- Sufficient IP addresses in each subnet

4 CLUSTER CREATION TOOLS

eksctl

- Simple YAML based configuration
- Quick cluster creation
- Manages node groups and add-ons

AWS CLI

- Full control and flexibility
- Scriptable and automatable
- Ideal for advanced use cases and CI/CD

5 INFRASTRUCTURE AS CODE

Terraform

- Define EKS infrastructure declaratively
- Version controlled and repeatable
- Supports multi-environment deployments
- Easily integrates with CI/CD

6 CLUSTER CONFIGURATION

- Configure cluster endpoint (public or private)
- Enable control plane logs
- Configure authentication and access entries
- Set up IAM roles and RBAC mapping
- Install essential add-ons (VPC CNI, CoreDNS, kube-proxy)
- Configure tags, logging, and monitoring

7 PRODUCTION DEPLOYMENT WORKFLOW

1. PLAN

- Define requirements
- Design architecture
- Choose AZs and networking
- Estimate capacity and cost

2. PREPARE

- Create VPC and subnets
- Configure IAM roles
- Set up CI/CD and IaC repositories
- Prepare kubectl and AWS access

3. CREATE CLUSTER

- Use eksctl / AWS CLI / Terraform
- Create EKS control plane
- Create and configure node groups or Fargate
- Verify cluster status

4. CONFIGURE

- Configure access and RBAC
- Install add-ons
- Enable logging and monitoring
- Apply security best practices

5. DEPLOY

- Deploy workloads
- Validate applications
- Test scalability and resilience
- Monitor and optimize

BEST PRACTICES FOR PRODUCTION EKS CLUSTERS

- Use private endpoint
- Distribute across multiple AZs
- Enable automatic upgrades
- Apply least privilege access
- Monitor and log everything
- Regular backup and tests

VERIQTA

EKS NETWORKING ARCHITECTURE

1 AMAZON VPC CNI

- Primary networking plugin for Amazon EKS
- Assigns ENIs and IP addresses to Pods
- Supports native VPC networking
- High performance and scalable
- Required for Network Policies

2 POD NETWORKING

- Each Pod gets a routable IP address
- Pods can communicate with any VPC resource
- No NAT for Pod-to-Pod or Pod-to-Service
- Supports Kubernetes Network Policies

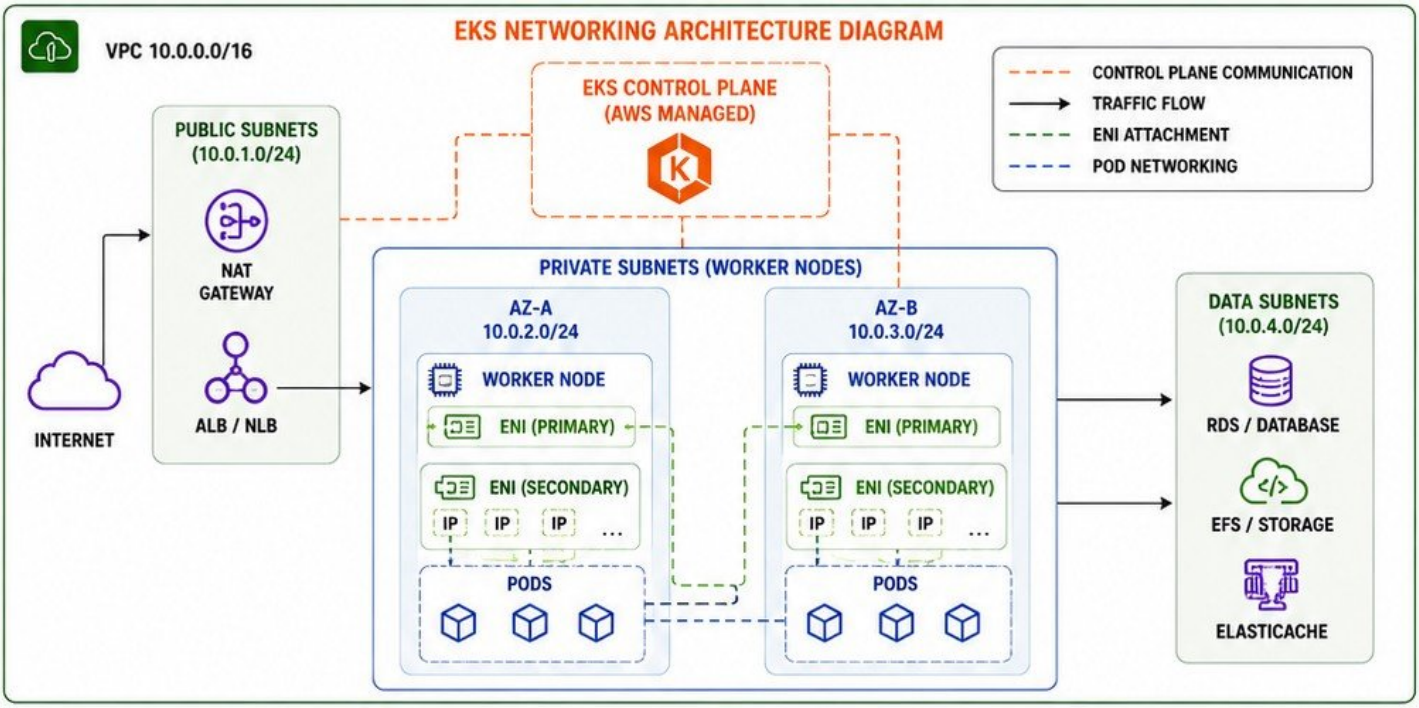
3 ENIs (ELASTIC NETWORK INTERFACES)

- ENIs are attached to worker nodes
- Each ENI provides multiple IP addresses
- Pods use IPs from these ENIs
- Primary ENI + Secondary ENIs

4 POD IP ALLOCATION

- IPs are allocated from subnet CIDR blocks
- Warm IP targets control pre-allocation
- Dynamic IP allocation for scalability
- Monitor available IPs to prevent exhaustion

EKS NETWORKING ARCHITECTURE DIAGRAM



5 PRIVATE AND PUBLIC SUBNETS

- Worker nodes
- Pods
- Databases
- Internal services

PUBLIC SUBNETS

- Load balancers
- NAT Gateways
- Bastion hosts (optional)
- Internet-facing services

6 SECURITY GROUPS

- Control traffic at the ENI level
- Nodes and Pods inherit security group rules
- Use least privilege rules
- Allow only required ports and CIDRs
- Combine with Network Policies for Pod-level protection

7 NETWORK TROUBLESHOOTING

- Pods cannot reach services or external resources
- Check Security Groups and Network Policies
- Verify route tables and subnet associations
- Check VPC CNI logs: /var/log/aws-routed-eni
- Use tools: kubectl, curl, nslookup, ping, traceroute

8 IP EXHAUSTION

- Insufficient IPs cause Pod scheduling failures
- Monitor available IPs in subnets
- Use larger subnets (/22 or bigger)
- Enable prefix delegation for more IPs per ENI
- Tune warm IP targets in VPC CNI

KEY BEST PRACTICES

- USE VPC CNI AND PREFIX DELEGATION
- MONITOR IP USAGE CONTINUOUSLY
- APPLY SECURITY GROUPS AND NETWORK POLICIES
- DESIGN SUBNETS WITH SUFFICIENT IP CAPACITY
- OBSERVE AND TROUBLESHOOT PROACTIVELY

VERIQTA

AMAZON EKS HANDBOOK

EKS IDENTITY AND ACCESS

1 AWS IAM

- Global AWS identity service
- Users, groups, and roles
- Policies define permissions
- Controls access to AWS resources
- Use IAM Identity Center for SSO integration

2 KUBERNETES RBAC

- Native Kubernetes access control
- Roles define permissions within the cluster
- RoleBindings grant roles to subjects
- Namespaced or cluster-wide access
- Use least privilege principle

3 EKS ACCESS ENTRIES

- New access management for EKS
- Map IAM identities to Kubernetes access
- Centralized and auditable
- Replaces aws-auth ConfigMap
- Supports fine-grained access control

4 ACCESS POLICIES

- AWS managed or custom policies
- Attach to IAM identities or roles
- Define what actions are allowed
- Use conditions for stronger controls
- Keep policies simple and purpose specific

5 IAM ROLES

- Roles are assumed by AWS services, users, or workloads
- Use roles instead of access keys
- Grant minimal permissions required
- Support cross-account access
- Use role chaining when needed

6 SERVICE ACCOUNTS

- Kubernetes identity for applications
- Used by Pods to access AWS resources
- Map to IAM roles using IRSA or Pod Identity
- One service account per application
- Avoid using default service account

7 LEAST PRIVILEGE

- Grant only the permissions that are required
- Apply at both AWS IAM and Kubernetes RBAC levels
- Regularly review and remove unused access
- Use conditions, resource scoping, and tags
- Least privilege reduces risk and blast radius

8 AUTHENTICATION VS AUTHORIZATION

AUTHENTICATION AUTHORIZATION

AUTHENTICATION

- Verifies who you are
- AWS IAM authenticates the identity
- EKS validates the identity

AUTHORIZATION

- Determines what you can do
- Kubernetes RBAC authorizes actions within the cluster
- AWS IAM authorizes actions on AWS resources

EKS IDENTITY AND ACCESS FLOW

USER / APPLICATION	AUTHENTICATION	ACCESS ENTRY	KUBERNETES RBAC	AWS RESOURCES	ACCESS GRANTED
User or Pod makes a request	AWS IAM authenticates the identity	EKS maps IAM identity using Access Entry	RBAC authorizes access to Kubernetes resources	IAM role authorizes access to AWS resources	Request is allowed if authorized

KEY BEST PRACTICES

- USE IAM IDENTITY CENTER (SSO)
- USE ACCESS ENTRIES INSTEAD OF aws-auth
- APPLY LEAST PRIVILEGE AT ALL LAYERS
- MAP SERVICE ACCOUNTS TO IAM ROLES (IRSA/POD ID)
- AUDIT AND REVIEW ACCESS REGULARLY
- MONITOR ACCESS AND ENABLE LOGGING

VERIQTA

IAM ROLES FOR SERVICE ACCOUNTS AND POD IDENTITY

1 EKS POD IDENTITY

- Simplified way for Pods to access AWS services
- No need for OIDC provider or IRSA setup
- Uses EKS Pod Identity Association
- Works with latest EKS versions
- Recommended approach for new deployments

2 IRSA (IAM ROLES FOR SERVICE ACCOUNTS)

IRSA

- Uses OIDC provider for authentication
- Associates IAM role with Kubernetes Service Account
- Pods assume IAM role using projected token
- Works on all supported EKS versions
- Ideal for fine-grained permission control

3 IAM ROLES

- IAM role defines AWS permissions
- Used by Pods to access AWS APIs
- Can be used with Pod Identity or IRSA
- Attach least privilege policies only
- Use role tags for organization and control

4 SERVICE ACCOUNTS

- Kubernetes identity for applications
- Linked to IAM role via Pod Identity or IRSA
- Namespace-scoped
- Enables secure access to AWS services
- Avoid using default service account

5 AWS API PERMISSIONS

- Define exact AWS actions and resources
- Example: s3:ListBucket, dynamodb:PutItem
- Avoid wildcard "*" where possible
- Use IAM policy conditions for extra control
- Follow least privilege best practices

6 CREDENTIAL ISOLATION

- Each Pod gets temporary, short-lived credentials
- Credentials are isolated per Pod
- No credential sharing between Pods
- Automatically rotated by AWS
- Reduces blast radius of compromise

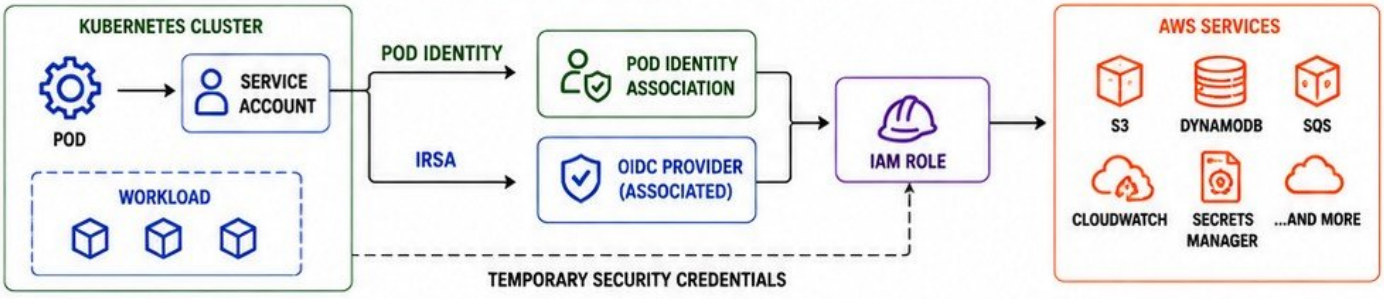
7 LEAST PRIVILEGE

- Grant only the permissions required
- Scope access to specific resources
- Regularly review and remove unused permissions
- Use condition keys and tags to restrict access
- Prinicple of least privilege reduces risk

8 TROUBLESHOOTING ACCESSDENIED ERRORS

- Check IAM role attached to Service Account
- Verify Pod Identity Association or IRSA setup
- Confirm IAM policy allows required action
- Check resource ARNs and conditions
- Use CloudTrail and AWS CLI for debugging

HOW PODS ACCESS AWS USING POD IDENTITY OR IRSA



KEY BEST PRACTICES

- USE POD IDENTITY FOR SIMPLICITY
- USE IRSA FOR FINE-GRAINED CONTROL
- APPLY LEAST PRIVILEGE IAM POLICIES
- MONITOR AND AUDIT ACCESS REGULARLY
- ROTATE AND REVIEW PERMISSIONS OFTEN
- TEST ACCESS AND TROUBLESHOOT EARLY

VERIQTA

EKS COMPUTE STRATEGY

1 MANAGED NODE GROUPS

- AWS manages EC2 instances for you
- Automatic scaling, patching, and updates
- Health monitoring and self-healing
- Integrates with Cluster Autoscaler and Karpenter
- Ideal for most production workloads

2 SELF-MANAGED NODES

- Full control over EC2 instances
- Custom AMIs, bootstrap, and configuration
- Use when you need special software or configurations
- Handle patching, scaling, and lifecycle
- More operational responsibility

3 AWS FARGATE

- Serverless compute for Pods
- No servers to manage
- Pay per vCPU and memory per second
- Great for spiky, short-lived, or microservices
- Works with Fargate Profiles or Pod-level selection

4 EC2 INSTANCE SELECTION

- Match instance type to workload needs
- Consider CPU, memory, network, and storage
- General purpose: M family
- Compute optimized: C family
- Memory optimized: R family
- Accelerated / GPU for ML workloads

5 SPOT INSTANCES

- Up to 90% cost savings
- Perfect for fault-tolerant and batch workloads
- Can be interrupted by AWS
- Use with Cluster Autoscaler or Karpenter
- Combine with On-Demand for stability

6 ON-DEMAND INSTANCES

- No interruptions
- Best for critical and stateful workloads
- Predictable performance and availability
- Higher cost compared to Spot
- Use for baseline capacity

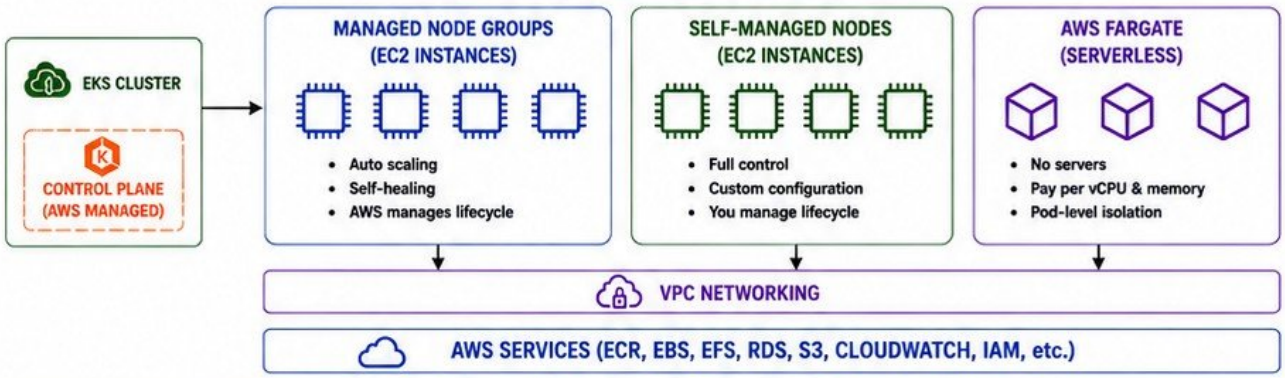
7 WORKLOAD PLACEMENT

- Use node labels and taints
- Node affinity / anti-affinity
- Topology spread constraints
- Separate workloads by environment or team
- Use Fargate for isolated or serverless workloads

8 COMPUTE TRADE-OFFS

- Cost vs reliability
- Control vs simplicity
- Performance vs flexibility
- Operational effort vs automation
- Choose the right mix for your workloads

EKS COMPUTE ARCHITECTURE OVERVIEW



COMPUTE STRATEGY COMPARISON

STRATEGY	MANAGED NODE GROUPS	SELF-MANAGED NODES	AWS FARGATE	SPOT INSTANCES	ON-DEMAND INSTANCES
MANAGEMENT	AWS MANAGED	YOU MANAGE	AWS MANAGED	AWS MANAGED	AWS MANAGED
COST	MEDIUM	MEDIUM	PAY PER USE (HIGHER)	LOWEST	HIGHEST
FLEXIBILITY	HIGH	HIGHEST	HIGH	HIGH	HIGH
BEST FOR	MOST WORKLOADS	SPECIAL REQUIREMENTS	SPIKY / MICROSERVICES	BATCH / FAULT-TOLERANT	CRITICAL / STATEFUL
DOWNTIME RISK	LOW	LOW	LOW	POSSIBLE INTERRUPTION	NONE

KEY BEST PRACTICES

- USE MANAGED NODE GROUPS FOR SIMPLICITY
- USE SPOT AND ON-DEMAND TOGETHER
- USE FARGATE FOR SPIKY WORKLOADS
- PLACE WORKLOADS WITH AFFINITY & TAINTS
- MONITOR UTILIZATION AND OPTIMIZE
- BALANCE COST, RELIABILITY & PERFORMANCE

VERIQTA EKS AUTOSCALING

1 HORIZONTAL POD AUTOSCALER (HPA)

- Scales the number of Pods based on metrics
- Uses CPU, memory or custom metrics
- Ensures application availability
- No changes to Pod configuration
- Works at Deployment, StatefulSet level

2 VERTICAL POD AUTOSCALER (VPA)

- Automatically adjusts CPU and memory requests/limits
- Recommends or updates resource settings
- Improves resource efficiency
- Reduces OOMKills and over-provisioning
- Works at Pod or Container level

3 CLUSTER AUTOSCALER

- Automatically adjusts the number of nodes in the cluster
- Adds nodes when Pods are pending
- Removes nodes when underutilized
- Works with Managed Node Groups or Self-Managed Nodes

4 KARPENTER

- Next-generation node provisioner
- Provisions right-sized nodes on demand
- Uses Spot and On-Demand capacity
- Faster provisioning and better bin packing
- Replaces Cluster Autoscaler or works together

5 METRICS

- CPU and memory metrics
- Custom application metrics
- Prometheus / CloudWatch as metrics source
- Metrics Server for resource metrics
- Accurate metrics = better scaling decisions

6 NODE PROVISIONING

- New nodes are launched to meet demand
- Uses Launch Templates and Auto Scaling Groups
- Subnets across multiple AZs
- Security groups and IAM roles applied
- Nodes become Ready and Pods are scheduled

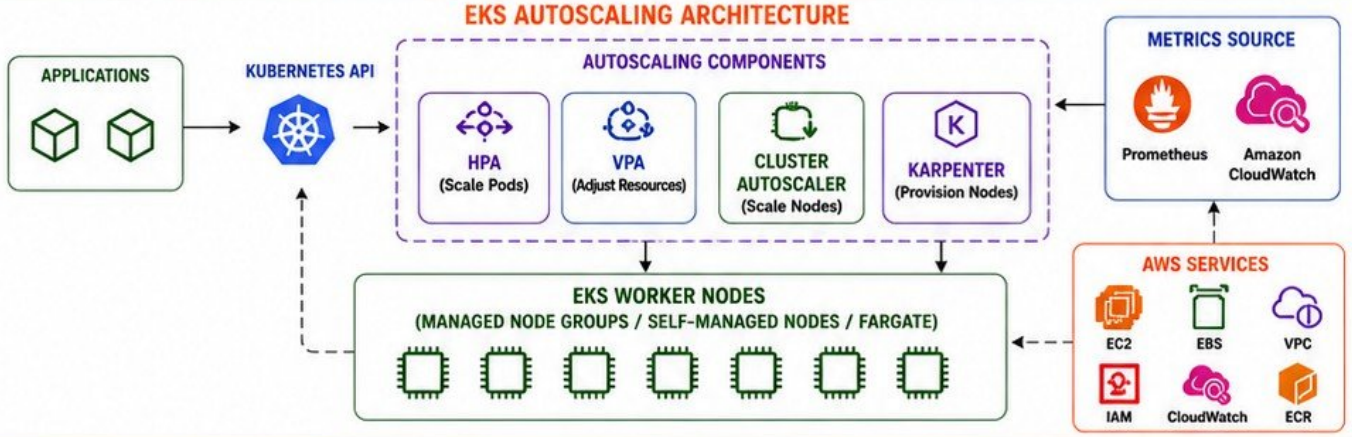
7 SCALING BOTTLENECKS

- Insufficient quotas (vCPU, ENI, IP addresses)
- Slow image pulls
- Resource limits too low
- Cluster Autoscaler cooldowns
- Pod affinity / anti-affinity constraints
- Single-AZ or limited capacity

8 PRODUCTION SCALING ARCHITECTURE

- Combine HPA + VPA + Cluster Autoscaler / Karpenter
- Use metrics-driven scaling
- Multi-AZ node groups
- Spot + On-Demand mix
- Monitor and optimize continuously

EKS AUTOSCALING ARCHITECTURE



AUTOSCALING SUMMARY

COMPONENT	WHAT IT SCALES	HOW IT WORKS	SCOPE	KEY BENEFIT
HPA	NUMBER OF PODS	BASED ON METRICS	DEPLOYMENT / STATEFULSET	HIGH AVAILABILITY
VPA	CPU & MEMORY	ADJUSTS REQUESTS/LIMITS	POD / CONTAINER	RESOURCE EFFICIENCY
CLUSTER AUTOSCALER	NUMBER OF NODES	ADDS / REMOVES NODES	CLUSTER	MATCH CAPACITY TO DEMAND
KARPENTER	NODES (RIGHT-SIZED)	PROVISIONS RIGHT NODES	CLUSTER	FASTER, CHEAPER, SMARTER

KEY BEST PRACTICES

- USE HPA FOR WORKLOAD DEMAND
- USE VPA IN RECOMMENDATION MODE FIRST
- ENABLE CLUSTER AUTOSCALER OR KARPENTER
- MONITOR METRICS AND SET ALERTS
- DISTRIBUTE NODES ACROSS MULTIPLE AZs
- USE SPOT INSTANCES FOR COST SAVINGS

VERIQTA

APPLICATION DEPLOYMENT ON EKS

1 DEPLOYMENTS

- Manages ReplicaSets and Pods
- Declarative updates
- Rolling updates and rollbacks
- Self-healing
- Desired state management
- Supports strategy configurations

2 REPLICASETS

- Ensures a specified number of Pod replicas
- Created and managed by Deployments
- Maintains stable Pods
- Handles scaling
- Pod replacement on failure

3 PODS

- Smallest deployable unit
- One or more containers per Pod
- Share IP address and network namespace
- Ephemeral and replaceable
- Use labels for grouping and selection

4 SERVICES

- Stable endpoint for Pods
- Service types: ClusterIP, NodePort, LoadBalancer, ExternalName
- Decouples traffic from Pod lifecycle
- Supports service discovery and load balancing

5 CONFIGMAPS

- Store non-sensitive configuration data
- Key-value pairs
- Mount as files or use as environment variables
- Decouple config from container image
- Update without rebuilding the image

6 SECRETS

- Store sensitive data securely
- Base64 encoded
- Mount as files or use as environment variables
- Use Kubernetes Secrets or external secret stores
- Restrict access with RBAC

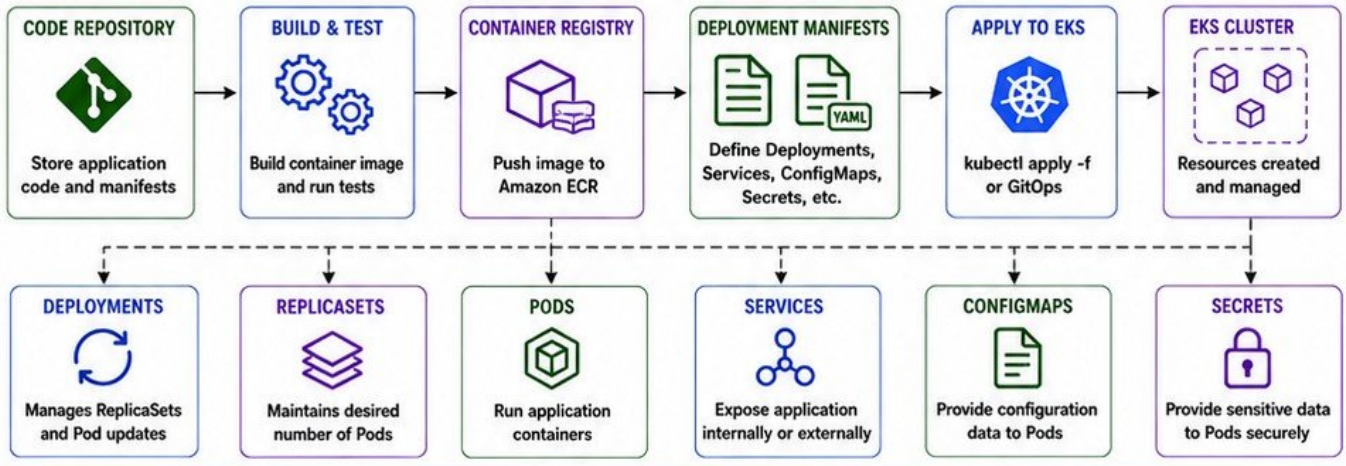
7 RESOURCE REQUESTS AND LIMITS

- Requests: minimum resources guaranteed
- Limits: maximum resources allowed
- CPU (millicores) and Memory (Mi/Gi)
- Prevents noisy neighbors
- Enables proper scheduling and autoscaling

8 BEST PRACTICES

- Use Deployments for stateless apps
- Use ConfigMaps and Secrets for configuration
- Set resource requests and limits for all containers
- Use liveness and readiness probes
- Monitor and set alerts

PRODUCTION DEPLOYMENT FLOW



DEPLOYMENT ESSENTIALS

RESOURCE	PURPOSE	EXAMPLE	KEY POINTS
DEPLOYMENTS	Manage application lifecycle	nginx-deployment	Rolling updates, rollbacks, self-healing
REPLICASETS	Maintain Pod replicas	nginx-rs	Ensures desired Pod count
PODS	Run application containers	nginx-pod	Ephemeral, replaceable
SERVICES	Provide stable network access	nginx-service	Load balancing, service discovery
CONFIGMAPS	Store application configuration	app-config	Non-sensitive key-value data
SECRETS	Store sensitive information	db-credentials	Passwords, tokens, keys
RESOURCE LIMITS	Control resource usage	cpu: 500m, memory: 512Mi	Prevents overuse, improves stability

VERIQTA

EKS LOAD BALANCING AND INGRESS

1 AWS LOAD BALANCER CONTROLLER

- Manages AWS Load Balancers for Kubernetes Services and Ingress
- Watches Services, Ingresses and annotations
- Creates and configures ALB or NLB automatically
- Supports HTTPS, TLS, WAF, redirects, and more
- Essential for production traffic management

2 APPLICATION LOAD BALANCER (ALB)

- Layer 7 load balancer
- HTTP and HTTPS traffic
- Path and host-based routing
- TLS termination
- Integrated with AWS WAF
- Ideal for web applications and microservices
- Supports WebSockets and gRPC

3 NETWORK LOAD BALANCER (NLB)

- Layer 4 load balancer
- TCP, UDP, and TLS traffic
- Ultra high performance and low latency
- Static IP support
- Ideal for high-throughput applications
- Used for non-HTTP services, databases, and gaming

4 KUBERNETES INGRESS

- Defines HTTP/HTTPS routing rules
- Routes traffic to Services based on host or path
- Works with Ingress Controller (ALB Controller)
- Centralized way to manage external access
- Supports TLS, redirects and rewrites

5 SERVICES

- Stable virtual IP and DNS for Pods
- Service types: ClusterIP, NodePort, LoadBalancer, ExternalName
- LoadBalancer type provisions an AWS Load Balancer
- Decouples clients from Pod lifecycle

6 TARGET GROUPS

- Target Group contains Pod IPs or Node IPs
- Health checks ensure only healthy targets receive traffic
- ALB: target type = ip (Pod IPs)
- NLB: target type = ip or instance
- Automatically managed by Load Balancer Controller

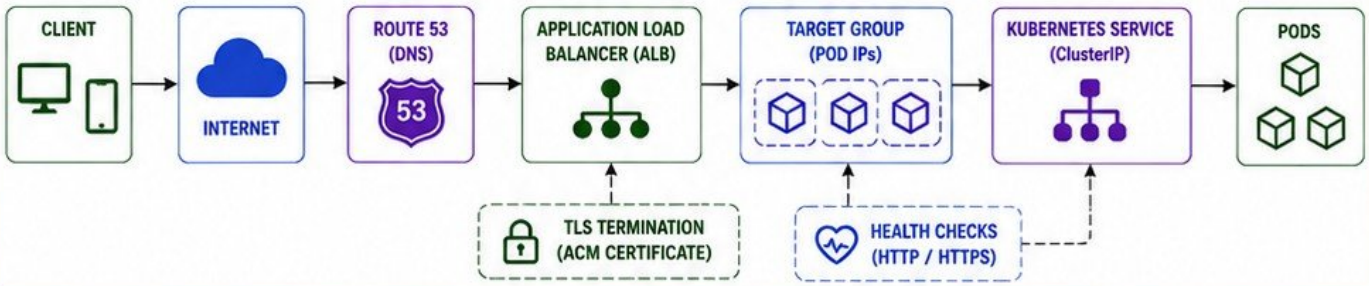
7 TLS TERMINATION

- Offloads SSL/TLS at the Load Balancer
- Reduces load on applications
- Supports ACM certificates
- End-to-end encryption optional
- Supports multiple certificates (SNI)

8 BEST PRACTICES

- Use ALB for HTTP/HTTPS workloads
- Use NLB for TCP/UDP workloads
- Enable TLS everywhere
- Use health checks
- Use WAF with ALB
- Monitor and set alerts
- Use least privilege IAM roles

EKS LOAD BALANCING REQUEST FLOW (ALB EXAMPLE)



SERVICE TYPE AND LOAD BALANCING BEHAVIOR

SERVICE TYPE	EXPOSED TO	USES LOAD BALANCER	USE CASE	EXAMPLE
ClusterIP	Internal (within cluster)	No	Internal communication	Microservices talking to each other
NodePort	External (via Node IP:Port)	No	Simple external access	Testing, small setups
LoadBalancer	External (via AWS LB)	Yes (ALB or NLB)	Production external access	Heath apps, APIs, services
ExternalName	External (DNS)	No	Map to external DNS name	Third-party services

KEY ANNOTATIONS FOR ALB INGRESS

ANNOTATION	PURPOSE
alb.ingress.kubernetes.io/scheme	internet-facing or internal
alb.ingress.kubernetes.io/listen-ports	Define HTTP/HTTPS ports
alb.ingress.kubernetes.io/certificate-arn	ACM certificate for TLS
alb.ingress.kubernetes.io/ssl-redirect	Redirect HTTP to HTTPS
alb.ingress.kubernetes.io/target-type	ip or instance

ANNOTATION	PURPOSE
alb.ingress.kubernetes.io/healthcheck-path	Health check endpoint
alb.ingress.kubernetes.io/healthcheck-protocol	HTTP or HTTPS
alb.ingress.kubernetes.io/healthcheck-port	Health check port
alb.ingress.kubernetes.io/group.name	Group multiple ingresses
alb.ingress.kubernetes.io/wafv2-acl-arn	Attach AWS WAF

VERIQTA EKS STORAGE

1 EBS CSI DRIVER

- Provides block storage for Kubernetes workloads
- Supports gp2, gp3, io1, io2, st1, sc1 volumes
- Dynamic provisioning of EBS volumes
- Supports snapshots and encryption
- Ideal for databases and low-latency workloads

2 EFS CSI DRIVER

- Provides shared file storage
- Accessed via NFS protocol
- Dynamic provisioning of EFS file systems
- Supports Access Points and IAM authorization
- Ideal for shared content, config, and big data

3 PERSISTENTVOLUMES (PV)

- Cluster-wide storage resource
- Backed by EBS, EFS, or other storage systems
- Defined by storage admin
- Independent of Pod lifecycle
- Static or dynamically provisioned

4 PERSISTENTVOLUMECLAIMS (PVC)

- Storage request by a user
- Binds to a suitable PV
- Requests size, access mode, and StorageClass
- Lives in a Namespace
- Decouples storage use from underlying storage details

5 STORAGECLASSES

- Defines storage type and provisioner
- Controls how PVs are provisioned
- Supports parameters like IOPS, throughput, encryption
- Enables dynamic provisioning of volumes

6 STATEFUL WORKLOADS

- Require stable storage
- Use PVCs for data persistence
- Examples: Databases, Kafka, ElasticSearch, Redis
- Use StatefulSets for stable identity and storage
- Data survives Pod restarts and rescheduling

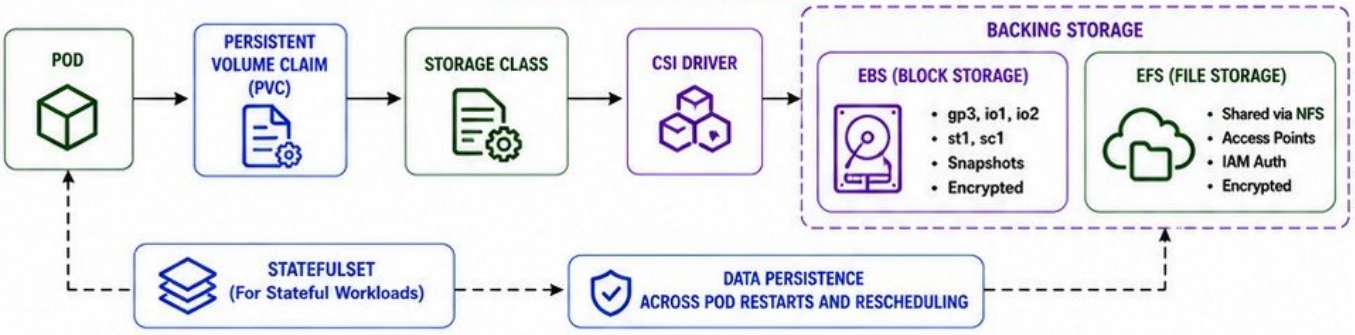
7 VOLUME LIFECYCLE

- Provisioned (static or dynamic)
- Bound to PVC
- Mounted to Pod
- Used by application
- Released (when PVC deleted)
- Reclaimed (Retain / Delete / Recycle policy)

8 STORAGE FAILURE TROUBLESHOOTING

- Pending PVCs (check StorageClass, provisioner, quotas)
- Volume mount failures (check events, permissions)
- EBS attach/detach issues (check node and AZ)
- EFS mount issues (NFS connectivity, security groups)
- Use kubectl describe, events, and CSI driver logs

EKS STORAGE ARCHITECTURE



STORAGE OPTIONS COMPARISON

STORAGE TYPE	PROTOCOL	ACCESS MODE	PERFORMANCE	USE CASES	DYNAMIC PROVISIONING
EBS (gp3 / io1 / io2)	Block (iSCSI)	RWO	High	Databases, OLTP, Critical Apps	Yes (via CSI Driver)
EBS (st1 / sc1)	Block (iSCSI)	RWO	Throughput Optimized	Big Data, Data Lakes, Archives	Yes (via CSI Driver)
EFS	File (NFS)	RWX	Scales with Throughput	Shared Data, Web Content, CI/CD	Yes (via CSI Driver)
FFS for Lustre	File (Lustre)	RWX	Very High	HPC, Machine Learning	Yes (via CSI Driver)

COMMON STORAGE ISSUES AND FIXES

ISSUE	POSSIBLE CAUSE	TROUBLESHOOTING STEPS	FIX
PVC Pending	No matching StorageClass or quota	kubectl describe pvc, check events	Create/ fix StorageClass, check quotas
Volume Mount Failed	Permission / path / driver issue	Check Pod events, CSI driver logs	Fix permissions, check driver, paths
EBS Attach Failed	AZ mismatch / volume in use	Check node AZ, volume attachments	Use correct AZ, detach old volume
EFS Mount Timeout	Security group / NFS / DNS issue	Test NFS connectivity, check SGs	Allow NFS (2049), fix SG/DNS
Pod Stuck in ContainerCreating	Storage not attached or mounted	kubectl describe pod, check events	Fix storage, restart Pod

VERIQTA
EKS SECURITY HARDENING

1 LEAST PRIVILEGE

- Grant only what is necessary
- Use IAM roles with minimal permissions
- Use Kubernetes RBAC for fine-grained access
- Regularly review and remove unused access
- Follow least privilege for humans and workloads

2 POD SECURITY STANDARDS

- Enforce Kubernetes Pod Security Standards
- Baseline: Minimum restrictions
- Restricted: Strong security controls
- Privileged: Not allowed in production
- Apply at namespace level using labels

3 SECURITY GROUPS

- Use SGs to control traffic to nodes
- Allow only necessary ports and sources
- Separate security groups for control plane and nodes
- Deny all by default
- Regularly audit and remove open access

4 NETWORK POLICIES

- Control Pod-to-Pod and namespace traffic
- Use deny-all by default policy
- Allow only required communication
- Micro-segmentation for applications
- Enforce with Calico, Cilium, or AWS VPC CNI

5 SECRETS PROTECTION

- Store secrets in AWS Secrets Manager or SSM Parameter Store
- Use Kubernetes Secrets only for non-sensitive data
- Encrypt secrets at rest and in transit
- Limit access with IAM and RBAC
- Rotate secrets regularly

6 IMAGE SECURITY

- Use trusted base images
- Scan images for vulnerabilities
- Use Amazon ECR with image scanning
- Sign images (e.g., Cosign)
- Block unsigned or untrusted images
- Keep images minimal and up to date

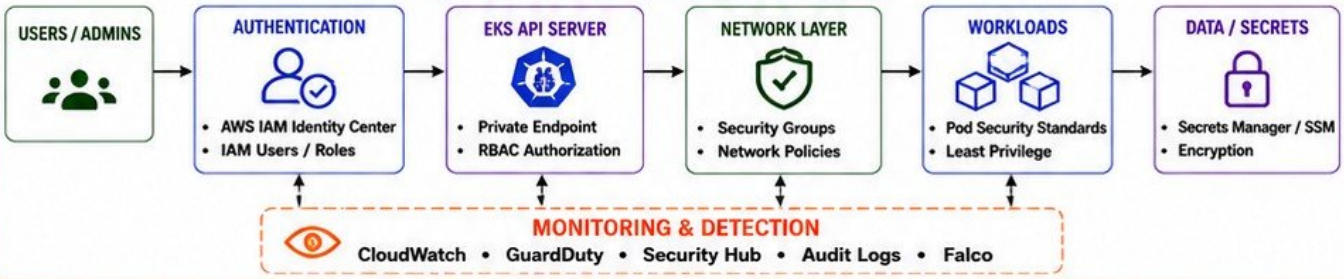
7 PRIVATE CLUSTERS

- Enable private endpoint only or private + public
- Restrict API server access to trusted networks
- Use Bastion or VPN for administration
- Enable PrivateLink for AWS services
- Keep worker nodes in private subnets

8 BEST PRACTICES

- Keep EKS, nodes, and add-ons up to date
- Enable audit logging
- Monitor with CloudWatch and GuardDuty
- Use Falco or AWS Runtime Security
- Perform regular security assessments
- Follow defense in depth

EKS SECURITY ARCHITECTURE OVERVIEW



POD SECURITY STANDARDS

LEVEL	DESCRIPTION
BASELINE	Minimally restrictive. Prevents known privilege escalations.
RESTRICTED	Default for production workloads. Disallows privilege escalation.
PRIVILEGED	Unrestricted. Should be avoided in production.

COMMON SECURITY CONTROLS

CONTROL	PURPOSE
LEAST PRIVILEGE	Limit access to resources
RBAC	Control Kubernetes access
NETWORK POLICIES	Control traffic between Pods
SECURITY GROUPS	Control traffic to nodes
ENCRYPTION	Protect data at rest and in transit
AUDIT LOGGING	Track and audit all actions

RISK VS MITIGATION

RISK	MITIGATION
OVER-PERMISSIONED ACCESS	Apply least privilege
EXPOSED API SERVER	Use private endpoint
UNRESTRICTED POD ACCESS	Enforce network policies
VULNERABLE IMAGES	Scan and update images
SECRETS EXPOSURE	Use Secrets Manager / SSM
LACK OF VISIBILITY	Enable monitoring and logs

PRODUCTION EKS SECURITY CHECKLIST

- ☑ Use private endpoint for EKS API server
- ☑ Enable IAM authentication and RBAC
- ☑ Enforce Pod Security Standards (Restricted)
- ☑ Apply least privilege IAM roles
- ☑ Enable network policies with deny-all default
- ☑ Restrict security groups (least open ports)
- ☑ Use IRSA for workloads
- ☑ Store secrets in AWS Secrets Manager / SSM
- ☑ Encrypt data at rest and in transit
- ☑ Scan container images on push and pull
- ☑ Enable audit logging for EKS
- ☑ Monitor with CloudWatch and GuardDuty
- ☑ Keep EKS, nodes, and add-ons updated
- ☑ Use private subnets for nodes
- ☑ Enable VPC Flow Logs
- ☑ Back up etcd (EKS managed)
- ☑ Test incident response and recovery
- ☑ Review and audit access regularly

VERIQTA
EKS OBSERVABILITY

METRICS • LOGS • TRACES • EVENTS

1 AMAZON CLOUDWATCH

- Centralized monitoring for EKS and AWS resources
- Alarms and dashboards
- Logs, metrics and events
- Anomaly detection and insights
- Integration with Container Insights

2 CONTAINER INSIGHTS

- Deep visibility into containers and nodes
- CPU, memory, disk, network metrics
- Performance monitoring
- Out-of-the-box dashboards
- Optimized for EKS workloads

3 PROMETHEUS

- Powerful metrics collection
- Kubernetes native monitoring
- Service discovery
- Alertmanager integration
- Long-term metrics storage
- Custom metrics and recording rules

4 GRAFANA

- Visualize metrics from multiple sources
- Pre-built Kubernetes dashboards
- Custom dashboards and panels
- Alerting and notifications
- Correlate metrics with logs and traces

5 APPLICATION LOGS

- Collect logs from containers and pods
- Structured logging (JSON)
- Use Fluent Bit / Fluentd or CloudWatch Agent
- Centralized log storage
- Search, filter and analyze logs easily

6 CONTROL PLANE LOGS

- Audit Logs
- Authenticator Logs
- Controller Manager Logs
- Scheduler Logs
- Enable via CloudWatch Logs
- Essential for security and troubleshooting

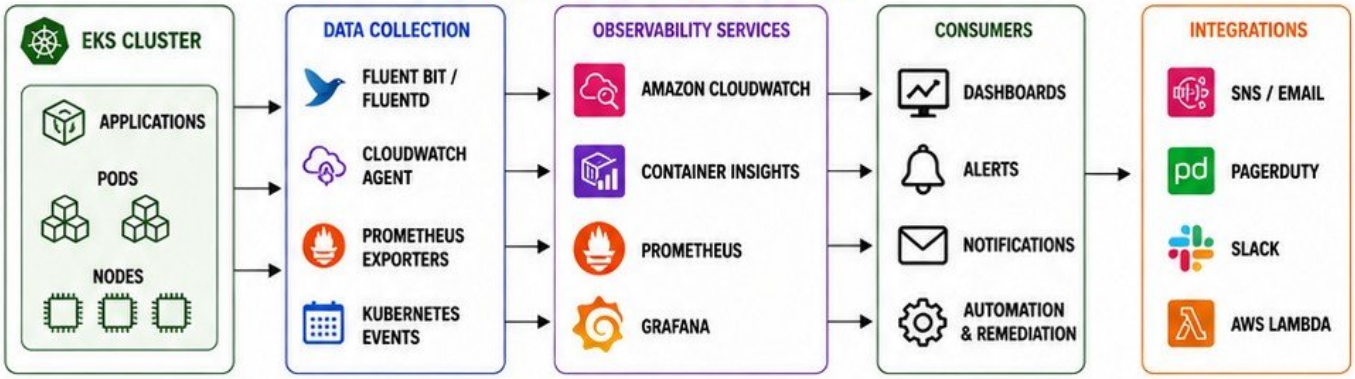
7 KUBERNETES EVENTS

- Real-time cluster events
- Pod lifecycle events
- Scheduling and scaling events
- View with kubectl or CloudWatch
- Helps in fast root cause analysis

8 METRICS, LOGS AND TRACES

- Metrics: Understand cluster and app health
- Logs: Investigate issues in detail
- Traces: Follow requests across services
- Correlate all three for complete observability

EKS OBSERVABILITY ARCHITECTURE



SECURE BY DESIGN ✓ IAM LEAST PRIVILEGE ✓ ENCRYPTION IN TRANSIT & AT REST ✓ PRIVATE NETWORKING

OBSERVABILITY BEST PRACTICES

- ✓ Enable Container Insights for all EKS clusters
- ✓ Centralize all logs in CloudWatch Logs
- ✓ Use structured logging for applications
- ✓ Create meaningful dashboards and alarms
- ✓ Monitor control plane logs for security
- ✓ Define SLOs and track error budgets
- ✓ Correlate metrics, logs and traces
- ✓ Regularly review and optimize observability

ESSENTIAL OBSERVABILITY CHECKLIST

- ☑ CloudWatch and Container Insights enabled
- ☑ Control plane logs enabled and monitored
- ☑ Application logs collected and searchable
- ☑ Prometheus scraping cluster and app metrics
- ☑ Grafana dashboards and alerts configured
- ☑ Kubernetes events monitored
- ☑ Alarms and notifications configured
- ☑ Runbooks and dashboards tested regularly

YOU CAN'T IMPROVE WHAT YOU DON'T OBSERVE. OBSERVE • UNDERSTAND • IMPROVE

VERIQTA

AMAZON EKS HANDBOOK

EKS PRODUCTION TROUBLESHOOTING

IDENTIFY • ANALYZE • RESOLVE • PREVENT

1 PENDING PODS

- Pod is accepted but not assigned to a node
- Check resources, scheduling, and quotas
- Common causes: insufficient resources, taints, affinity, PVC issues

KEY COMMANDS

```
kubectl get pods -A
kubectl describe pod <pod>
kubectl get events -A --sort-by=.lastTimestamp
```

2 CRASHLOOPBACKOFF

- Container starts then crashes repeatedly
- Check logs, config, resource limits
- Common causes: app errors, bad config, missing dependencies

KEY COMMANDS

```
kubectl describe pod <pod>
kubectl logs <pod> -c <container> --previous
kubectl get events --field-selector involvedObject.name=<pod>
```

3 IMAGEPULLBACKOFF

- Cannot pull container image from registry
- Check image name, registry access and secrets
- Common causes: wrong image, auth, network

KEY COMMANDS

```
kubectl describe pod <pod>
kubectl get secrets
kubectl logs <pod> -c <container>
```

4 OOMKILLED

- Container terminated due to out of memory
- Check memory limits and usage
- Common causes: memory leak, insufficient limits

KEY COMMANDS

```
kubectl describe pod <pod>
kubectl top pod <pod>
kubectl top node <node>
```

5 FAILEDSCHEDULING

- Scheduler cannot place pod on any node
- Check node resources, constraints, and policies
- Common causes: taints, node selectors, affinity

KEY COMMANDS

```
kubectl describe pod <pod>
kubectl get nodes --show-labels
kubectl get events --field-selector reason=FailedScheduling
```

6 NODE NOT READY

- Node is unhealthy or not communicating
- Check node status, network, kubelet
- Common causes: network issues, disk pressure, kubelet stopped

KEY COMMANDS

```
kubectl get nodes
kubectl describe node <node>
kubectl get events --field-selector involvedObject.name=<node>
```

7 DNS FAILURES

- Pods cannot resolve service or external names
- Check CoreDNS, svc records, and network
- Common causes: CoreDNS down, network policies, VPC DNS issues

KEY COMMANDS

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl describe pod -n kube-system -l k8s-app=kube-dns
kubectl exec -it <pod> -- nslookup <service>
```

8 OTHER COMMON ISSUES

- PVC Pending
- Service Unavailable
- High Latency
- 5xx Errors
- Check events, logs, metrics, and configs

KEY COMMANDS

```
kubectl get events -A --sort-by=.lastTimestamp
kubectl logs -l app=<app> --tail=100
kubectl get all -A
```

KUBECTL TROUBLESHOOTING WORKFLOW

1 IDENTIFY THE PROBLEM

- Understand the symptoms
- Check alerts and user reports

COMMAND

```
kubectl get pods -A
```

2 GATHER INFORMATION

- Describe resources
- Check events
- Check logs

COMMANDS

```
kubectl describe pod <pod>
kubectl get events -A
kubectl logs <pod>
```

3 CHECK METRICS

- Review resource usage
- Check node and pod metrics

COMMANDS

```
kubectl top pod -A
kubectl top node
```

4 ANALYZE ROOT CAUSE

- Correlate logs, events, metrics
- Identify misconfig or failure

TOOLS

CloudWatch
Container Insights
Prometheus / Grafana

5 RESOLVE THE ISSUE

- Fix configuration
- Restart or redeploy
- Scale or adjust resources

COMMANDS

```
kubectl rollout restart deploy/<app>
kubectl apply -f <file.yaml>
```

6 VERIFY AND PREVENT

- Verify service recovery
- Add alerts and dashboards
- Document and improve

COMMANDS

```
kubectl get all -A
kubectl get events -A
```

TROUBLESHOOTING CHECKLIST

- ☑ Check Pod status and events
- ☑ Review container logs
- ☑ Verify resource requests and limits
- ☑ Check node status and conditions
- ☑ Validate network connectivity and DNS
- ☑ Review ConfigMaps, Secrets and volumes
- ☑ Check security groups and network policies
- ☑ Correlate with metrics and dashboards
- ☑ Identify root cause and fix
- ☑ Verify recovery and monitor

USEFUL KUBECTL COMMANDS

`kubectl get pods -A`	List all pods in all namespaces
`kubectl describe pod <pod>`	Show details and events for a pod
`kubectl logs <pod> -c <container> --tail=200`	View container logs
`kubectl get events -A --sort-by=.lastTimestamp`	Show recent events
`kubectl top pod -A`	Show pod resource usage
`kubectl top node`	Show node resource usage
`kubectl get nodes`	List nodes and their status
`kubectl describe node <node>`	Show details and conditions of a node
`kubectl exec -it <pod> -- /bin/sh`	Access a running container shell
`kubectl get all -A`	List all resources in all namespaces

OBSERVE DEEPLY. UNDERSTAND QUICKLY. FIX ACCURATELY. PREVENT CONTINUOUSLY.

VERIQTA

AMAZON EKS HANDBOOK

EKS NETWORKING TROUBLESHOOTING

IDENTIFY • DIAGNOSE • RESOLVE • VERIFY

1 POD-TO-POD FAILURES

- Pods cannot communicate with each other
- Check network policies, security groups, and routing
- Verify CNI plugin and IP allocation

KEY CHECKS

```
kubectl exec -it <pod> -- ping <pod-ip>
kubectl exec -it <pod> -- traceroute <pod-ip>
kubectl get netpol -A
kubectl describe pod <pod>
```

2 POD-TO-SERVICE FAILURES

- Pods cannot reach ClusterIP/NodePort/LoadBalancer
- Verify service endpoints
- Check kube-proxy and iptables
- Validate routing and network policies

KEY CHECKS

```
kubectl get svc,ep -A
kubectl exec -it <pod> -- curl <svc-ip>:<port>
kubectl describe svc <svc>
kubectl logs -n kube-system -l k8s-app=kube-proxy
```

3 DNS RESOLUTION ISSUES

- Pods cannot resolve service names or external domains
- Check CoreDNS and DNS configuration
- Verify nodes and VPC DNS settings

KEY CHECKS

```
kubectl exec -it <pod> -- nslookup kubernetes.default
kubectl exec -it <pod> -- nslookup google.com
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl describe configmap -n kube-system coredns
```

4 COREDNS ISSUES

- DNS pods are crashing or not healthy
- Check CoreDNS logs, config and resources
- Verify upstream resolvers and network access

KEY CHECKS

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
kubectl describe pod -n kube-system <coredns-pod>
kubectl -n kube-system edit configmap coredns
```

5 SECURITY GROUPS ISSUES

- Traffic blocked by node or cluster security groups
- Verify inbound and outbound rules
- Ensure required ports are open

KEY CHECKS

```
aws ec2 describe-security-groups
kubectl get nodes -o wide
kubectl describe node <node>
Check SG rules for: 443, 10250, 179,
30000-32767 (NodePort)
```

6 NETWORK POLICIES ISSUES

- NetworkPolicy may be blocking traffic between pods
- Check default deny policies
- Validate namespace selectors and rules

KEY CHECKS

```
kubectl get netpol -A
kubectl describe netpol -n <ns> <netpol>
kubectl get pods -n <ns> -o wide
Test connectivity from allowed
and denied pods
```

7 VPC CNI FAILURES

- Pods stuck in ContainerCreating
- IP allocation failures
- ENI or prefix delegation issues
- CNI plugin errors

KEY CHECKS

```
kubectl -n kube-system logs -l k8s-app=aws-node
kubectl -n kube-system describe daemonset aws-node
kubectl get pods -o wide
kubectl describe pod <pod>
Check CNI metrics in CloudWatch
```

8 IP EXHAUSTION INVESTIGATION

- No available IPs for pods
- Check ENI and IP usage
- Increase subnet **CIDR**, use prefix delegation
- Clean up unused ENIs

KEY CHECKS

```
kubectl get pods -A -o wide | wc -l
aws ec2 describe-network-interfaces
  --filters "Name=tag:eks:cluster-name"
Check CloudWatch: awscni-available-ip-address-count
Enable prefix delegation for more IPs per ENI
```

NETWORK TROUBLESHOOTING WORKFLOW

1. IDENTIFY THE ISSUE	2. GATHER INFORMATION	3. ISOLATE THE LAYER	4. RUN DIAGNOSTIC TESTS	5. FIND ROOT CAUSE	6. RESOLVE THE ISSUE	7. VERIFY AND PREVENT
Understand the symptoms and impact	Collect data, logs, events and metrics	Pod, Service, DNS, Node, VPC or Security	Use kubectl, logs, metrics and AWS tools	Confirm the root cause of the failure	Apply fix and validate the solution	Verify connectivity and implement monitoring

ESSENTIAL TOOLS AND COMMANDS

- kubectl (logs, exec, describe, get, top)
- aws cli (VPC, EC2, Security Groups, ENIs)
- VPC Flow Logs
- CloudWatch Metrics and Logs
- CoreDNS logs and metrics
- kube-proxy logs
- tcpdump / traceroute / curl / nslookup

COMMON PORTS AND PROTOCOLS

- HTTPS (443) - EKS API Server
- 10250 - Kubelet API (Nodes)
- 179 - BGP (VPC CNI)
- 53 - DNS (UDP/TCP)
- 30000-32767 - NodePort Range
- 10256 - kube-proxy metrics

NETWORKING BEST PRACTICES

- ☑ Use private subnets for nodes and pods
- ☑ Keep security groups least privilege
- ☑ Use Network Policies to control traffic
- ☑ Monitor IP usage and plan capacity
- ☑ Enable VPC Flow Logs
- ☑ Use prefix delegation for better IP efficiency
- ☑ Regularly review connectivity and policies

OBSERVE THE NETWORK. UNDERSTAND THE FLOW. FIX THE ROOT CAUSE. PREVENT THE NEXT INCIDENT.

VERIQTA EKS UPGRADES

PLAN • TEST • UPGRADE • VERIFY • MONITOR

1 KUBERNETES VERSION LIFECYCLE

- EKS supports multiple Kubernetes versions
- Each version has a support window
- Upgrade before a version reaches end of support
- Review deprecation notices and release notes

KEY ACTIONS

- Check version lifecycle regularly
- Plan upgrades before end of support
- Review What's New in Kubernetes
- Understand breaking changes

2 CONTROL PLANE UPGRADES

- Upgrade the control plane first
- EKS manages control plane upgrade
- Minimal downtime within an AZ
- Verify cluster after upgrade

KEY ACTIONS

- Review version compatibility
- Upgrade control plane
- Validate cluster and add-ons
- Monitor control plane logs

3 NODE UPGRADES

- Nodes must be at the same or newer version
- Drain nodes before upgrading
- Upgrade node groups or self-managed nodes
- Verify workloads after node upgrade

KEY ACTIONS

- Check kubectl get nodes -o wide
- Drain node: kubectl drain <node>
- Upgrade node or node group
- Uncordon node: kubectl uncordon <node>

4 ADD-ON COMPATIBILITY

- Add-ons must be compatible with the target version
- Includes VPC CNI, CoreDNS, kube-proxy, EBS CSI, etc.
- Check AWS compatibility matrix
- Upgrade add-ons after control plane upgrade

KEY ACTIONS

- Review add-on versions
- Upgrade add-ons as needed
- Validate add-on functionality
- Monitor for errors

5 MANAGED NODE GROUP UPGRADES

- EKS rolls out new nodes with the new version
- Old nodes are drained and replaced
- Supports rolling or blue/green strategy
- Minimize disruption to workloads

KEY ACTIONS

- Choose update strategy
- Set max unavailable %
- Monitor upgrade progress
- Validate workloads

6 POD DISRUPTION BUDGETS

- Protect applications during upgrades
- Define minAvailable or maxUnavailable
- Ensures high availability during node drains
- Required for production workloads

KEY ACTIONS

- Create PDBs for all critical apps
- Validate PDBs before upgrade
- Check PDB status during upgrade
- Adjust if workloads are blocked

7 UPGRADE TESTING

- Test upgrades in a non-prod environment
- Validate application functionality
- Run integration and performagce tests
- Test add-ons and custom integrations

KEY ACTIONS

- Create staging cluster
- Perform upgrade end-to-end
- Run automated tests
- Document results and issues

8 SAFE PRODUCTION UPGRADE STRATEGY

- Plan, test, and communicate
- Upgrade in phases
- Monitor and rollback if necessary
- Validate workloads and SLOs
- Keep clear rollback plan ready

KEY ACTIONS

- Communicate upgrade window
- Follow phased approach
- Monitor metrics and logs
- Rollback if issues detected

EKS UPGRADE WORKFLOW

1. **ASSESS** — Review current version, support lifecycle, and compatibility matrix.
2. **PLAN** — Plan upgrade window, backup, and rollback strategy.
3. **TEST** — Test upgrade in staging and validate applications.
4. **UPGRADE CONTROL PLANE** — Upgrade the EKS control plane.
5. **UPGRADE NODES AND ADD-ONS** — Upgrade managed node groups, nodes, and add-ons.
6. **VALIDATE** — Verify workloads, monitor metrics, and check logs.
7. **MONITOR AND OPTIMIZE** — Monitor performance, SLOs, and optimize post-upgrade.

COMMON UPGRADE CHECKLIST

- [x] Review Kubernetes version lifecycle and release notes
- [x] Verify control plane and node version compatibility
- [x] Ensure add-ons are compatible with target version
- [x] Create and validate Pod Disruption Budgets
- [x] Back up etcd (if self-managed) and critical data
- [x] Test upgrade in staging environment
- [x] Communicate upgrade window to all teams
- [x] Upgrade control plane
- [x] Upgrade node groups or node AMIs
- [x] Upgrade add-ons
- [x] Validate workloads and critical paths
- [x] Monitor metrics, logs, and events
- [x] Document outcome and lessons learned

UPGRADE BEST PRACTICES

- [x] Always upgrade control plane before nodes
- [x] Keep nodes at the same or newer version than control plane
- [x] Use managed node groups for easier upgrades
- [x] Use rolling or blue/green strategies for node upgrades
- [x] Set appropriate maxUnavailable during upgrades
- [x] Use PDBs to maintain application availability
- [x] Monitor CloudWatch metrics and EKS events
- [x] Have rollback plan and tested backups
- [x] Upgrade during off-peak hours
- [x] Regularly review AWS EKS documentation and best practices

UPGRADES ARE INEVITABLE. PREPARATION IS OPTIONAL. DOWNTIME IS NOT.

VERIQTA

HIGH AVAILABILITY AND RESILIENCE

BUILD RESILIENT SYSTEMS • SURVIVE FAILURES • MAINTAIN AVAILABILITY

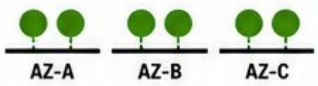
1 MULTI-AZ WORKER NODES

- Spread nodes across multiple AZs
- No single AZ is a single point of failure
- Provides high availability at the infrastructure level

KEY ACTIONS

- Create node groups in each AZ
- Use subnets in different AZs
- Verify node distribution
- Monitor AZ capacity

2 POD TOPOLOGY SPREAD



- Distribute pods across failure domains (AZs, nodes, racks)
- Prevents all replicas from running in one AZ or node
- Uses topologySpreadConstraints

KEY ACTIONS

- Define topology spread rules
- Use topology.kubernetes.io/zone
- Set maxSkew and whenUnsatisfiable
- Validate distribution

3 ANTI-AFFINITY

- Ensure pods do not run on the same node or AZ
- Prevents correlated failures
- Use podAntiAffinity rules

KEY ACTIONS

- Define podAntiAffinity rules
- Use preferredDuringScheduling or requiredDuringScheduling
- Combine with topology spread

4 POD DISRUPTION BUDGETS

- Limit the number of pods that can be down voluntarily
- Protects availability during upgrades and maintenance
- Ensures minimum availability

KEY ACTIONS

- Create PDB for each critical app
- Set minAvailable or maxUnavailable
- Verify PDB before upgrades
- Monitor distruptions

5 HEALTH PROBES

- Detect unhealthy containers early
- Kubernetes can restart or remove unhealthy pods
- Types: Liveness, Readiness, Startup probes

KEY ACTIONS

- Configure all three probes
- Set correct initial delays
- Use realistic thresholds
- Test failure scenarios

6 REPLICA STRATEGY

- Run multiple replicas for high availability
- More replicas = better resilience and load handling
- Use Deployments or StatefulSets

KEY ACTIONS

- Set appropriate replica count
- Distribute replicas across AZs
- Monitor replica health
- Use HPA for scaling

7 NODE FAILURE

- Kubernetes reschedules pods on healthy nodes
- Use multiple nodes per AZ
- Monitor node health continuously

KEY ACTIONS

- Enable Cluster Autoscaler
- Configure node health checks
- Use taints and tolerations
- Test node failure recovery

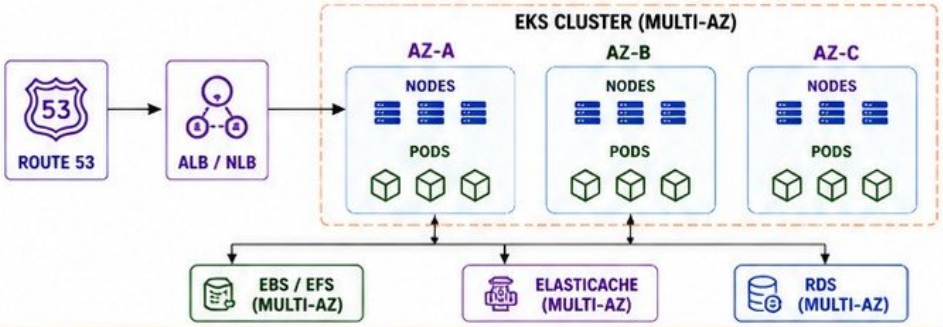
8 AZ FAILURE

- Design for full AZ failure
- Traffic and workloads continue in healthy AZs
- Requires multi-AZ nodes and subnets

KEY ACTIONS

- Spread nodes across 3 AZs
- Distribute workloads evenly
- Test AZ outage scenarios
- Use multi-AZ load balancers

HIGH AVAILABILITY ARCHITECTURE EXAMPLE



RESILIENCE BENEFITS

- NO SINGLE POINT OF FAILURE
- WORKLOADS STAY AVAILABLE DURING FAILURES
- AUTOMATIC RECOVERY AND RESCHEDULING
- MAINTAINS USER EXPERIENCE AND SLAs
- SUPPORTS PLANNED AND UNPLANNED DISRUPTIONS

BEST PRACTICES

- [x] Deploy worker nodes in at least 2–3 AZs
- [x] Use topology spread and anti-affinity rules
- [x] Configure Pod Disruption Budgets for critical apps
- [x] Use health probes on all containers
- [x] Run multiple replicas and monitor health
- [x] Test failure scenarios regularly
- [x] Automate recovery and scaling
- [x] Document runbooks for failures

FAILURE SCENARIOS AND EXPECTED BEHAVIOR

SCENARIO	IMPACT	EXPECTED BEHAVIOR
NODE FAILURE	One or more pods affected	Pods rescheduled on healthy nodes
AZ FAILURE	All workloads in that AZ down	Workloads continue in remaining AZs
POD FAILURE	Single pod down	Replica restarted automatically
UPGRADE / MAINTENANCE	Pod eviction	PDB ensures minimum availability
NETWORK PARTITION	Partial connectivity	Traffic routed to healthy endpoints

HA CHECKLIST

- [x] Multi-AZ worker nodes
- [x] Topology spread configured
- [x] Anti-affinity rules applied
- [x] Pod Disruption Budgets created
- [x] Health probes configured
- [x] Replica count set appropriately
- [x] Load balancer is multi-AZ
- [x] Critical data stores are multi-AZ
- [x] Monitoring and alerts enabled
- [x] Failure scenarios tested

VERIQTA
EKS CI/CD AND GITOPS

1 GITHUB ACTIONS

- Automate build, test, and deployment
- Native Kubernetes and AWS support
- Reusable workflows and actions
- Secure with OIDC and least privilege

2 CODEPIPELINE

- Fully managed CI/CD service
- Integrates with AWS services
- Visual pipeline stages
- Approval actions and notifications

3 ARGO CD

- GitOps continuous delivery
- Declarative and automated
- Self-healing applications
- Detects and syncs drift
- Web UI, CLI, and API

4 CONTAINER REGISTRY

- Store and manage container images
- Version and tag images
- Scan for vulnerabilities
- Integrate with CI/CD pipelines

5 AMAZON ECR

- Fully managed container registry by AWS
- High availability and durability
- IAM integration
- Image scanning with Amazon Inspector

6 MANIFEST DEPLOYMENT

- Deploy Kubernetes manifests
- Helm charts or Kustomize
- Store manifests in Git
- Promote across environments
- Immutable and repeatable

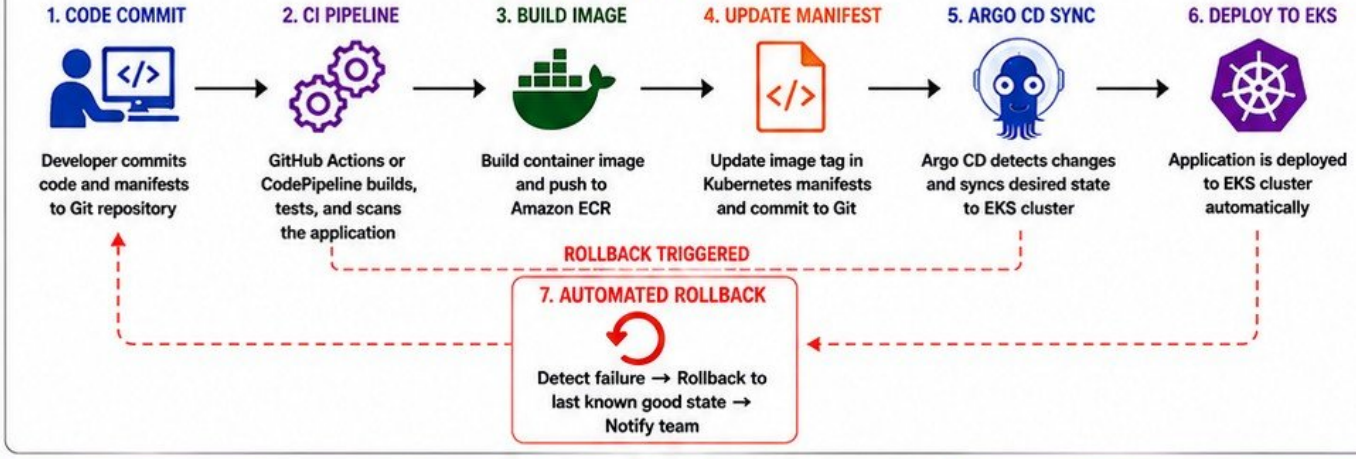
7 AUTOMATED ROLLBACK

- Detect failed deployments
- Rollback automatically
- Use Argo CD or pipeline rollback actions
- Maintain application stability
- Notify and create alerts

8 GITOPS PRINCIPLES

- Git is the single source of truth
- Declarative desired state
- Automated synchronization
- Audit-friendly history
- Secure and consistent

GITOPS PRODUCTION FLOW



PIPELINE TOOLS COMPARISON

TOOL	TYPE	BEST FOR	KEY BENEFIT
GITHUB ACTIONS	CI/CD	Custom workflows	Flexible and extensible
CODEPIPELINE	CI/CD	AWS native pipelines	Fully managed
ARGO CD	GITOPS	Kubernetes deployments	Automated GitOps
AMAZON ECR	REGISTRY	Store container images	Secure and integrated

BEST PRACTICES

- Store all manifests in Git
- Use semantic versioning for images
- Scan images and dependencies
- Use least privilege IAM roles
- Automate approvals for production
- Enable monitoring and alerts
- Test rollback regularly
- Keep environments consistent

KEY BENEFITS

- Fast and reliable deployments
- Consistent and repeatable releases
- Audit trail and visibility
- Self-healing and drift detection
- Reduced manual errors
- Faster recovery with rollback
- Secure and scalable by design

GIT + AUTOMATION + OBSERVABILITY = RELIABLE EKS DEPLOYMENTS

VERIQTA

EKS COST AND PERFORMANCE

1 NODE RIGHTSIZING

- Choose the right instance type and size
- Match node capacity to workload needs
- Avoid over-provisioned nodes
- Regularly review and adjust node sizes

2 POD REQUESTS AND LIMITS

- Set CPU and memory requests for all containers
- Set limits to prevent noisy neighbor issues
- Helps scheduler pack pods efficiently
- Prevents overuse and improves stability

3 SPOT CAPACITY

- Use Spot Instances for cost savings
- Ideal for stateless and fault-tolerant workloads
- Combine with On-Demand for critical workloads
- Monitor Spot interruption and use PDBs

4 KARPENTER CONSOLIDATION

- Karpenter provisions right-sized nodes automatically
- Consolidates underutilized nodes
- Drains and removes idle nodes safely
- Reduces waste and lowers costs

5 IDLE RESOURCES

- Identify underutilized nodes and pods
- Stop unused workloads
- Remove unused cluster add-ons
- Use Cluster Autoscaler or Karpenter consolidation

6 COMPUTE UTILIZATION

- Monitor CPU, memory, and pod capacity
- Track node and pod utilization over time
- Aim for 60%–80% utilization
- Avoid consistently low or high utilization

7 COST VISIBILITY

- Enable AWS Cost Explorer and Cost & Usage Report
- Use CloudWatch and Container Insights
- Tag resources and workloads
- Track cost by team, environment, and app

8 PERFORMANCE VS COST DECISIONS

- Understand workload performance needs
- Choose instance types that balance cost and performance
- Use right storage, network, and caching
- Optimize for business outcomes, not just cost

EKS COST OPTIMIZATION STRATEGY

ANALYZE WORKLOADS AND USAGE → RIGHTSIZE NODES AND PODS → USE SPOT AND KARPENTER → ELIMINATE IDLE RESOURCES → MONITOR AND TRACK COSTS → OPTIMIZE CONTINUOUSLY BALANCE PERFORMANCE AND COST

COST OPTIMIZATION TECHNIQUES

TECHNIQUE	WHAT IT DOES
RIGHTSIZE NODES	Match node size to workload needs
SET POD REQUESTS	Improve scheduling and resource packing
USE SPOT INSTANCES	Lower compute cost for compatible workloads
KARPENTER CONSOLIDATION	Remove underutilized nodes automatically
REMOVE IDLE RESOURCES	Stop unused workloads and resources
OPTIMIZE STORAGE	Use right storage class and lifecycle policies
MONITOR CONTINUOUSLY	Track usage, performance, and cost trends

KEY METRICS TO MONITOR

METRIC	WHAT TO WATCH
NODE CPU UTILIZATION	Keep between 60% and 80%
NODE MEMORY UTILIZATION	Keep between 60% and 80%
POD COUNT PER NODE	Avoid over-packing nodes
POD CPU / MEMORY REQUEST %	Low requests lead to poor packing
CLUSTER COST PER DAY	Track trend and optimize
SPOT INTERRUPTION RATE	Keep within acceptable limits
IDLE NODES	Should be minimal or zero

TARGET UTILIZATION RANGE

60%	60% – 80%	80%+
LOWER THAN 60%	OPTIMAL RANGE	HIGHER THAN 80%
WASTING RESOURCES AND MONEY	BEST BALANCE OF COST AND PERFORMANCE	RISK OF PERFORMANCE AND STABILITY ISSUES

COST VISIBILITY TOOLS

AWS COST EXPLORER | COST & USAGE REPORT | CLOUDWATCH CONTAINER INSIGHTS | AWS CUR AND TAGS

RIGHT SIZE. RIGHT COST. RIGHT PERFORMANCE.
CONTINUOUSLY OPTIMIZE FOR MAXIMUM VALUE.

VERIQTA
EKS PRODUCTION INCIDENT

1 APPLICATION BECOMES UNAVAILABLE

- Users report errors or timeouts
- Alerts fire (CloudWatch, PagerDuty)
- Confirm impact and severity
- Note symptoms and timestamps

2 IDENTIFY BLAST RADIUS

- Identify affected services and namespaces
- Determine impacted users and regions
- Check recent deployments or changes
- Assess dependencies and integrations

3 INSPECT PODS AND NODES

- Check pod status and restarts
- Inspect node health and readiness
- Look for CrashLoopBackOff, ImagePullBackOff, OOMKilled
- Verify resource usage

4 CHECK EVENTS

- Review Kubernetes events
- Look for scheduling failures, service issues, or errors
- Identify warnings and failures
- Correlate event timestamps

5 ANALYZE LOGS AND METRICS

- Check application logs
- Review container logs
- Analyze metrics (CPU, Memory, Errors, Latency)
- Correlate alerts and dashboards

6 VALIDATE NETWORKING

- Test pod-to-pod connectivity
- Test pod-to-service connectivity
- Check Ingress, Services, Endpoints
- Verify DNS resolution
- Review Network Policies, SGs, NACLs

7 IDENTIFY ROOT CAUSE

- Correlate findings from logs, metrics, events, and networking
- Identify the exact failure point
- Validate the root cause
- Document what happened

8 RECOVER SERVICE

- Fix the configuration or code
- Redeploy or rollback as needed
- Restart pods or scale if required
- Validate service health
- Monitor until fully stable

9 PREVENT RECURRENCE

- Implement permanent fix
- Add alerts and dashboards
- Improve tests and CI/CD checks
- Update runbooks and documentation
- Conduct a post-incident review

EKS INCIDENT RESPONSE WORKFLOW

ALERT TRIGGERED → ASSESS IMPACT → INVESTIGATE DEEP → RESOLVE ISSUE → RECOVER & VALIDATE → PREVENT & IMPROVE

KEY PRINCIPLES

- ACT FAST
- INVESTIGATE WITH DATA
- VERIFY EVERY FIX
- COMMUNICATE CLEARLY
- IMPROVE CONTINUOUSLY

ESSENTIAL COMMANDS CHEAT SHEET

PURPOSE	COMMAND
GET PODS	kubectl get pods -A
POD DETAILS	kubectl describe pod <pod-name> -n <ns>
GET NODES	kubectl get nodes
NODE DETAILS	kubectl describe node <node-name>
GET EVENTS	kubectl get events -A --sort-by=.metadata.creationTimestamp
POD LOGS	kubectl logs <pod-name> -n <ns>
PREVIOUS LOGS	kubectl logs <pod-name> -n <ns> --previous
TOP PODS (USAGE)	kubectl top pods -A
TOP NODES (USAGE)	kubectl top nodes
CHECK SERVICES	kubectl get svc -A
CHECK ENDPOINTS	kubectl get endpoints -A
CHECK INGRESSES	kubectl get ingress -A

QUICK INCIDENT CHECKLIST

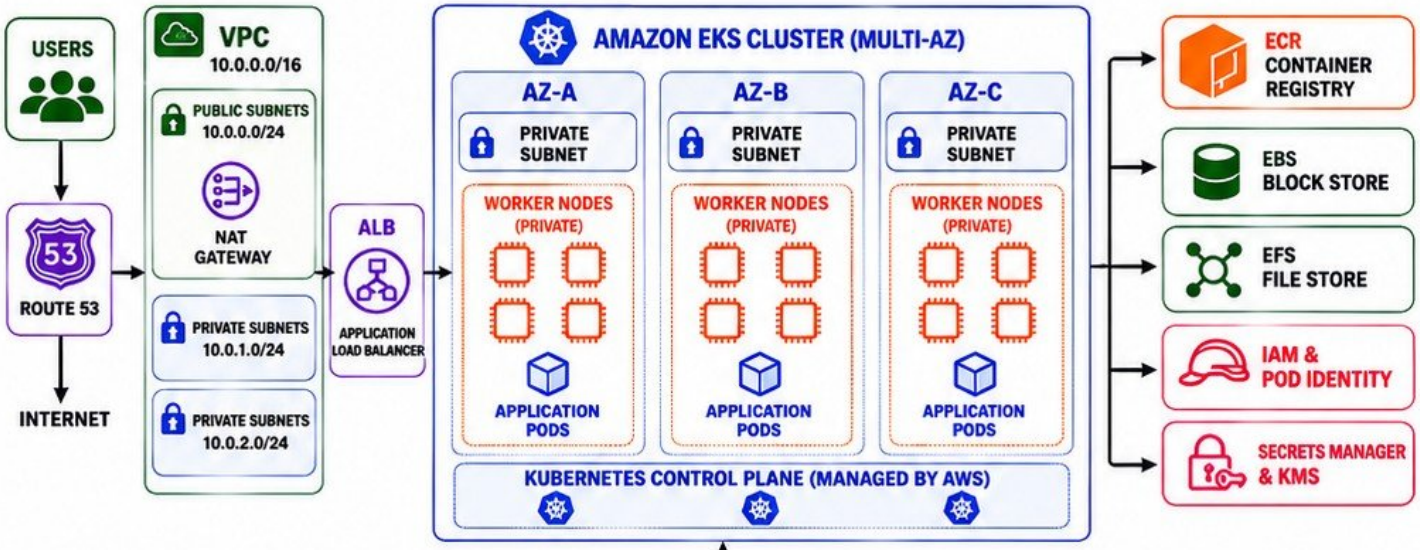
- ☑ Confirm alerts and impact
- ☑ Identify affected namespaces
- ☑ Check recent changes
- ☑ Inspect pods and nodes
- ☑ Review Kubernetes events
- ☑ Analyze logs and metrics
- ☑ Validate networking
- ☑ Identify root cause
- ☑ Implement fix or rollback
- ☑ Validate service recovery
- ☑ Monitor until stable
- ☑ Communicate resolution
- ☑ Document incident
- ☑ Update alerts and runbooks
- ☑ Perform post-incident review
- ☑ Improve to prevent recurrence

FAST DETECTION. DEEP INVESTIGATION. PERMANENT RESOLUTION.
BUILD RESILIENT EKS SYSTEMS.

VERIQTA COMPLETE PRODUCTION EKS ARCHITECTURE

KEY COMPONENTS

ROUTE 53 · APPLICATION LOAD BALANCER · MULTI-AZ EKS CLUSTER · PRIVATE WORKER NODES · ECR · EBS & EFS STORAGE · IAM & POD IDENTITY · KARPENTER AUTO PROVISIONING · CLOUDWATCH LOGS & METRICS · PROMETHEUS & GRAFANA MONITORING · CI/CD & GITOPS · SECRETS & SECURITY CONTROLS



KARPENTER AUTO PROVISIONING · CLOUDWATCH LOGS & METRICS · PROMETHEUS & GRAFANA MONITORING · CI/CD PIPELINE (CODEPIPELINE OR GITHUB ACTIONS) · GITOPS (ARGO CD)

COMPLETE REQUEST FLOW

1. USER → 2. ROUTE 53 → 3. ALB → 4. POD → 5. DATA LAYER

1. User resolves domain via Route 53
2. Request goes to Application Load Balancer
3. ALB routes traffic to healthy pods in the EKS cluster
4. Pods process request and return response
5. Application reads/writes data from EBS or EFS

FAILURE AND RECOVERY PATHS

- POD FAILURE (CrashLoop, OOMKilled) → KUBERNETES SELF-HEALING (Restart / Reschedule) → NEW POD STARTED ON HEALTHY NODE
- NODE FAILURE (Unhealthy) → EKS DETECTS & DRAINS NODE → PODS RESCHEDULED TO HEALTHY NODES (KARPENTER PROVISIONS NEW NODES)
- AZ FAILURE (Outage) → TRAFFIC REROUTED TO HEALTHY AZs VIA ALB → APPLICATION REMAINS AVAILABLE

ARCHITECTURE HIGHLIGHTS

- Multi-AZ EKS cluster for high availability
- Private worker nodes in private subnets
- ALB in public subnets for secure ingress
- ECR for secure container image management
- EBS & EFS for resilient storage
- IAM & Pod Identity for least privilege access
- Karpenter for intelligent auto-scaling
- CloudWatch for logs, metrics and alarms
- Prometheus & Grafana for deep observability
- CI/CD pipeline for automated delivery
- GitOps with Argo CD for declarative deployments
- Secrets Manager & KMS for secrets protection

PRODUCTION BEST PRACTICES

- Use private subnets for worker nodes
- Enable IAM roles for service accounts
- Keep images in Amazon ECR
- Use Karpenter for cost optimization
- Monitor with CloudWatch and Prometheus
- Store secrets in Secrets Manager

FINAL OUTCOME

- High availability
- Secure by design
- Cost optimized
- Auto healing
- Production ready

COMPLETE EKS PRODUCTION: SECURE • SCALABLE • RESILIENT